SOC L1 LAB · INC-002

Detection, Mitigation, and Triage Guide — INC-002

Catching privilege assignment, not just account creation

INCIDENT	INC-002 — Rogue local account added to sudo
REPORT DATE	2026-10-31
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Detection model
2. Rule-by-rule
3. Tuning
4. Triage playbook
5. Severity matrix
6. Containment and mitigation
7. Escalation template
8. Post-incident
9. Common mistakes
10. Coverage map
11. Metrics

Audience: an L1 analyst who receives the alert, and an L2 or detection engineer who owns the rule.

This guide uses INC-002 as the worked example. Rule IDs are Wazuh 4.14.7 stock unless marked custom.

---

1. Detection model

Account creation produces three independent signals. Each answers a different question.

Layer	Source	Question	INC-002 evidence
Identity	shadow-utils / PAM	Did a user or group appear?	5901, 5902
Process	auditd execve	Which command created it?	80792 `useradd`, `chpasswd`, `usermod`
File	FIM + audit watches	Which files changed?	550 on `/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow`

None of them, alone, says **the account was given root capability**. That is what the custom rule adds.

1.1 Signal hierarchy

```
Low     5902  new user, level 8
        5901  new group, level 8
        ----  inventory event; easy to defer

Medium  550   identity file changed
        80792 useradd / usermod executed
        ----  context; who and what

High    100121 account added to sudo
        account added to wheel/admin
        sudo used by a new or service account
        ----  privilege assignment; investigate now
```

---

2. Rule-by-rule

2.1 Rule 5902 — new user

- **Level:** 8
- **Data:** `dstuser`, `uid`, `gid`, `home`, `shell`
- **INC-002:** fired at 04:30:48.947 for `svc-ops`, uid 1003.
- **Use:** confirms the object. It does not grade the privilege.

2.2 Rule 5901 — new group

- **Level:** 8
- **INC-002:** fired at 04:30:48.942.
- **Use:** supporting context for the account's private group. A new privileged group is a different finding and should be treated as such.

2.3 Rules 550 / 554 — FIM on identity files

- **Levels:** 7 (checksum changed), 5 (file added)
- **INC-002:** `/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/gshadow` modified within one second; lock files added.
- **Use:** proves the identity database changed and gives hashes. It does not say which account or group changed, unless `report_changes` diff output is enabled.
- **Limitation:** noisy on any account or package change; do not alert on every 550.

2.4 Rules 80781 / 80792 — audit watches and commands

- **Level:** 3
- **80781** reports write access to `/etc/group` or `/etc/passwd` with the executable.
- **80792** reports the command. INC-002 captured the full argument vector:

```
{ "exe": "/usr/sbin/usermod",
  "execve": { "a0": "usermod", "a1": "-aG", "a2": "sudo", "a3": "svc-ops" } }
```

This is the field that makes a precise rule possible. The earlier `bash -c` event in INC-001 dropped `a2`; short, conventional commands retain it.

2.5 Rule 100121 — custom, privilege assignment

```
<rule id="100121" level="12">
  <if_sid>80792</if_sid>
  <field name="audit.exe">/usr/sbin/usermod</field>
  <field name="audit.execve.a2">sudo</field>
  <description>Privilege assignment: user added to the sudo group ($(audit.execve.a3))</description>
  <mitre>
    <id>T1098</id>
    <id>T1548.003</id>
  </mitre>
  <group>audit_command,privilege_escalation,</group>
</rule>
```

- **Fires on:** `usermod -aG sudo <user>` and `usermod -G sudo <user>`.
- **INC-002:** 04:30:48.390, naming `svc-ops`.
- **False positives:** legitimate onboarding and automated provisioning. Match them to a change ticket; do not suppress the rule.
- **Limitations:** hard-codes the group `sudo`; depends on auditd; does not catch a direct `/etc/group` edit.

Test:

```
sudo useradd -m -s /bin/bash svc-test
sudo usermod -aG sudo svc-test
# expect 100121 naming svc-test within ~1 second
sudo gpasswd -d svc-test sudo && sudo userdel -r svc-test
```

---

3. Tuning

3.1 Privileged groups

`sudo` is Debian's default. Other estates use `wheel` (RHEL family), `admin`, or a custom group. Options:

- duplicate the rule for each group (simple, explicit), or
- use a CDB list of privileged groups and match `audit.execve.a2` against it.

A list is better if the set changes. Test it with `wazuh-logtest` before deployment.

3.2 Direct edits to `/etc/group`

An attacker who can write `/etc/group` directly bypasses `usermod` and therefore 100121. Cover this with:

- FIM 550 on `/etc/group` (already present), and
- a membership baseline, checked on a schedule or at alert time:

```
getent group sudo | sort > /var/lib/soc/sudo-group.baseline
# periodic
getent group sudo | sort | diff -u /var/lib/soc/sudo-group.baseline -
```

A drift alert is low-noise and catches edits that never touch `usermod`.

3.3 Change-control context

The rule cannot know whether a change was approved. Two practical options:

1. Require the ticket ID in the command or in a wrapper, e.g. a provisioning script that records `CHG-1234` in the audit trail.
2. Enrich the alert with a lookup against the change system before escalation.

Without one of these, every legitimate onboarding becomes a manual review.

3.4 Service-account policy

Detection is the backstop; the policy is the control:

- service accounts get `/usr/sbin/nologin`, not `/bin/bash`;
- no password, keys only, and only where the service needs to authenticate;

- never in `sudo`;
- no home directory unless the service requires one;
- a naming convention that is enforced, not just suggested.

3.5 Audit coverage

- Keep `audit-wazuh-c` for execve; it is what captures the argument vector.
- Keep `audit-wazuh-w` on `/etc/passwd`, `/etc/group`, `/etc/shadow`, `/etc/sudoers`.
- Do not enable every syscall; 160 command alerts in 80 seconds is already a lot.

---

4. Triage playbook

4.1 The five questions

1. **Who created or changed it?** `auid`, `uid`, session, source.
2. **Was it authorised?** Change ticket, window, naming convention, owner.
3. **What privilege did it get?** `sudo`, `wheel`, `admin`, sudoers entry.
4. **Has it been used?** `sudo` events, SSH logins, `su`, cron.
5. **What else came with it?** SSH keys, cron, systemd units, shell profile changes.

4.2 Decision tree

```
Account/group change
│
├─ Approved change found
│   └─ Close with the ticket ID and the group baseline updated
│
└─ No approval
    ├─ Privileged group (sudo/wheel/admin)? → High; contain now
    ├─ Interactive shell + password?        → raise severity
    ├─ Account used (sudo/login)?           → Critical; scope the use
    └─ Persistence (key/cron/unit)?         → Critical; full response
```

4.3 Step by step

1. **Read the raw audit record.** Confirm the command, the target, and the group.
2. **Pull the identity files.** Note the hashes and the exact group line.
3. **Check the actor.** `auid` is the human; `uid=0` is expected after `sudo`.
4. **Check for use.** `journalctl` and the 5402/5715 alerts for the account.
5. **Check persistence before deletion.** Keys, cron, systemd, shell profiles.
6. **Assign severity** with the matrix in §5.
7. **Contain** in the right order: remove privilege, lock, kill sessions, delete.
8. **Preserve evidence** before deleting the home directory.
9. **Escalate** with the template in §7.

4.4 What a good ticket contains

- the account, UID, group, and shell;
- the exact commands and their timestamps;
- whether a change request exists;
- whether the account was used;
- the group membership before and after;
- the containment actions and times;
- the specific question for L2 or the change owner.

---

5. Severity matrix

Event	No use observed	Privileged use	Persistence
New account, no privileged group	Low	Medium	High
New account + `sudo` / `wheel`	**High**	Critical	Critical
Existing account added to privileged group	High	Critical	Critical
Privileged group membership changed by unknown actor	High	Critical	Critical

INC-002: new account + `sudo`, no use observed → **High**.

Raise one level if the account can authenticate remotely, if the host is a domain controller or jump host, or if the actor's session is suspected compromised.

---

6. Containment and mitigation

6.1 During the incident

Action	Command	Caution
Remove from the privileged group	`gpasswd -d svc-ops sudo`	Do this before deleting the account
Lock the account	`usermod -L svc-ops` / `chage -E 0`	Check service impact
Kill sessions	`pkill -KILL -u svc-ops`	Capture the process list first
Delete the account	`userdel -r svc-ops`	Only after checking for persistence
Check for keys	`find /home/svc-ops -name authorized_keys -o -name .ssh`	Do this before deleting the home
Check cron	`crontab -l -u svc-ops`, `/etc/cron.d`	User cron may not exist
Check units	`systemctl list-unit-files --state=enabled`	Compare to baseline
Preserve evidence	Copy alerts, journal, audit, `getent group`	Before any deletion

6.2 Host hardening

```
# service accounts should not have shells
sudo usermod -s /usr/sbin/nologin svc-ops

# service accounts should not be in sudo
sudo gpasswd -d svc-ops sudo

# lock an account without deleting it
sudo usermod -L svc-ops
sudo chage -E 0 svc-ops
```

6.3 Group baseline

```
getent group sudo  > /var/lib/soc/sudo-group.baseline
getent group wheel >> /var/lib/soc/sudo-group.baseline 2>/dev/null || true
getent group admin >> /var/lib/soc/sudo-group.baseline 2>/dev/null || true
```

Review on a schedule, and alert on any diff. This is the only control that catches a direct `/etc/group` edit without relying on a specific binary.

6.4 Policy controls

- Approval required for any local account or privileged group change.
- Service accounts: `nologin`, no password, no `sudo`.
- Periodic attestation of `sudo` / `wheel` membership.

- Onboarding and offboarding tied to the HR/IT workflow.
- Local admin actions reviewed monthly.

---

7. Escalation template

```
INC-002 — Rogue local account added to sudo

What:  At 04:30:46 UTC, `analyst` created local account `svc-ops` (uid 1003) with
       /bin/bash and a password, then added it to the `sudo` group at 04:30:47.
       No change request found. `sudo` membership is root capability on this host.
Evidence: 100121 (level 12) at 04:30:48, 5902/5901 at 04:30:48, FIM 550 on
       /etc/passwd, /etc/group, /etc/shadow, /etc/gshadow.
Use:   No sudo command or SSH login by svc-ops observed.
Done:  Removed from sudo, account deleted at 04:32:07; group back to analyst,debian.
Ask:   (1) Confirm whether `analyst` was authorised. (2) Check other hosts for the
       same account name. (3) Review sudo group membership estate-wide.
```

---

8. Post-incident

- Add the account creation and sudo assignment to the detection test suite.
- Measure: action to 100121 (about 2 seconds), action to 5902 (about 2 seconds).
- Confirm the `sudo` group baseline was updated after remediation.
- Review whether the actor's session shows other suspicious commands.
- Decide whether the account name pattern should be blocked for interactive shells.

---

9. Common mistakes

Mistake	Consequence
Closing on 5902 alone because the name looks like a service account	Misses the privilege assignment
Deleting the account before checking keys/cron	Destroys evidence and may miss a second foothold
Removing the account but not the group entry	The group line can remain until the account is deleted
Treating the source IP as identity	In this lab every session is the gateway
Suppressing legitimate onboarding	Better: match to a change ticket
Assuming `usermod` is the only path	Direct `/etc/group` edits bypass 100121
Ignoring the actor	An admin account is either the control or the threat

---

10. Coverage map

Technique	ID	Covered by	Gap
Create local account	T1136.001	5902, 80792	No change-window context
Account manipulation	T1098	100121, FIM 550	Direct file edits; custom groups
Abuse elevation control: sudo	T1548.003	100121, 5402	Use of an existing sudo account
Valid accounts	T1078	5715, 5402	Admin vs attacker indistinguishable by IP

11. Metrics

Metric	INC-002 value
Action to 100121	~2 s
Action to 5902	~2 s
Alerts in the window	237
Relevant account events	3 (100121, 5901, 5902)
Time to containment	~81 s after the action
Accounts created	1
Privileged commands by the account	0

The detection worked. The delay was in deciding whether the change was authorised, which is a process problem, not a sensor problem.